Training Run Monitor: final report

Repository: https://github.com/rauantuken06/Training-Run-Monitor · **Live app:** https://training-run-monitor-gold.vercel.app/ (Vercel, commit `20654d3`) · Soup Frontend Engineer take-home, November 2026

Overview and approach

The app shows a training-run dashboard for 20 deterministic synthetic runs (loss, `lr`, `grad_norm`), including one run with **1,000,000 points and a single spike at step 500,123 (loss 42.5)**. The guiding rule: the chart must never look cleaner than the data. Stack: React 19, TypeScript (strict), Vite 6, Tailwind 4, React Router 7, ECharts 6 (SVG renderer), Vitest, Playwright.

- **Pipeline:** generate → `analyzeRun` (integrity engine) → segment-aware **M4 downsampling** for the visible range and pixel width → one ECharts series per contiguous segment.
- **Off the main thread:** generation and analysis run in a **Web Worker**. It returns lossless typed-array columns plus a null mask, so NaN, ±Infinity, −0 and `null` all survive the transfer.
- **Lazy loading:** charts and ECharts are a separate chunk. Search, filters and sort live in the URL.

Data truthfulness guarantees (each enforced by tests)

- **Raw data is never mutated.** Normalization produces derived structures and reports every change in `DataQualityDiagnostics`.
- **`null`, `NaN` and ±Infinity stay distinct.** `null` means "not recorded"; NaN and ±Infinity mean "recorded but invalid". None is ever plotted as 0: each appears as a labelled marker (for example "NaN ×600") and a written warning.
- **Every plotted coordinate is an exact raw (step, value).** There is no smoothing, ECharts sampling or animation. Downsampling keeps the exact minimum and maximum of each pixel column, so the spike appears at every width and zoom level, which was checked against a brute-force reference.
- **Gaps and invalid values break the line**, never bridged. Conflicting duplicate steps are flagged, and the last recorded value is plotted.
- **The tooltip reports what is really under the cursor.** When one pixel spans many observations, it says so.

Tests and evidence

Summary with real output and skip reasons: `docs/test-evidence.md`.

Check	Result (final run, Windows 11)	Evidence
Lint / typecheck / build	pass / pass / pass (0 errors each)	`docs/qa/logs/{lint,typecheck,build}.log`
Unit and component tests (Vitest)	564 passed, 0 failed (35 files)	`docs/qa/logs/unit.log`
e2e Chrome 157 (desktop + Pixel 7 emulation)	86 passed, 8 skipped[1], 0 failed	`docs/qa/logs/e2e-chrome.log`
e2e Edge (desktop + mobile emulation)	86 passed, 8 skipped[1], 0 failed	`docs/qa/logs/e2e-edge.log`
e2e Firefox (Playwright build, desktop)	46 passed, 1 skipped[1], 0 failed	`docs/qa/logs/e2e-firefox.log`
axe WCAG 2.0–2.2 A/AA, light and dark	0 violations in every tested state	`e2e/final-qa.spec.ts`, `docs/final-qa-report.md`
`npm audit`	production deps: 0; dev-only: 7 moderate (lint tooling)	`docs/qa/logs/npm-audit.log`
Smoke test on the **live URL**, Chrome + Pixel 7 (12 tests)	1 worker: 12/12 (candidate), 12/12 (agent). 4 workers: 12/12, then **9/12**[4]	`docs/qa/logs/production-smoke-*.log`

[4] Timeouts; cause unconfirmed, not fixed. [1] Intentional skips: desktop-only keyboard and mouse-hover tests on the mobile project, and the missing-asset 404 check, which only applies to a deployment (`vite preview` serves `index.html` for every path). Key tests: the spike's drawn SVG vertex lies within 0.5 px of (500,123, 42.5) at full range and zoomed, and an exact hover shows "Step 500,123 · Loss: 42.5"; 400 seeded randomized integrity cases and 300 randomized downsampling cases; request races (A → B → C); 40 responsive screenshots in `docs/qa/screenshots/`.

Performance (Task 08, measured)

Production build, Chrome 157 at **4× CPU throttling** (Chrome DevTools Protocol), median of 5 runs. Traces: `docs/performance/{baseline,after}/traces/*.json.gz`.

Metric	Before	After
run-20 total blocking time (to charts painted)	3,638 ms	**1,023 ms** (−72%)
run-20 longest main-thread task	1,824 ms	**332 ms** (−82%)
run-20 charts painted (first render)[2]	4,348 ms	2,530 ms[2]
Runs list: table painted / JS downloaded	978 ms / 950 kB	763 ms / 371 kB
Zoom in, new viewport / revisited viewport	504 / 360 ms	440 / 144 ms
Reset zoom (first click)	648 ms	176 ms
Hover, input → next frame (median / max)	32 / 122 ms	31 / 122 ms (unchanged; not a bottleneck)

Metric	Before	After
run-20 tab memory (Task 09, Windows process counters)[3]	—	123–128 MB private, 223–225 MB peak while loading; JS heap 74.9 MB

[2] **CPU throttling does not slow the Web Worker** (verified), so this time is overstated for a slow CPU; with the worker slowed equally, charts would paint at about 3.8 s (**an estimate, not a measurement**). Main-thread figures are unaffected. [3] Windows counters of the tab's renderer process (page + worker; the worker's share cannot be separated, and no worker remains after loading). **Not** Chrome Task Manager's "Memory footprint" (not measured); no earlier baseline.

Silent failures found and fixed

Found in	Defect (reproduced first, then covered by a regression test)
Task 07	Range form read "5.000" as step 5 (now: whole steps only, `.` rejected)
Task 07	A requested range outside the data was silently replaced by a different one (now trimmed to the data and labelled "Adjusted to …"; a range with no data shows an error)
Task 07	The tooltip headline named a neighbouring step as the hovered one (now names the cursor step or the pixel's step range)
Task 07	run-20 `lr` y-axis printed six identical tick labels (precision now follows the visible range)
Task 09	**Fast typing in search dropped characters** ("loss" became "lss"; 5 of 5 runs at 6× throttling): the input followed an asynchronously updated URL
Task 09	Light-theme axis lines had 2.56:1 contrast (WCAG 1.4.11 needs 3:1; now 4.76:1). axe could not check the chart SVG.
Task 09	A worker that never answered left the page on "Loading…" forever (now a 60 s timeout with Retry)

Also checked with no defect found: request races, stale cache, error after success, reset restoring anomaly markers, diverging runs staying in range, zero and negative values, and fixed en-US number and UTC date formatting.

Accessibility and QA

- **Automated:** axe finds 0 violations (WCAG 2.0–2.2 A/AA) in light and dark mode on the list, run pages (details open, form error), unknown run and 404.
- **Checked by hand:** WCAG contrast of every chart colour, a visible 2 px focus outline on every Tab stop, a logical keyboard order, and reflow at 320 and 640 px (no sideways scrolling).
- **Gaps:** in Firefox, six axe rules are skipped on one state (they crash Playwright's Firefox build). Not tested: screen readers (none installed), real browser zoom, forced colours, Safari, real phones. **This is not an accessibility certification.**

Verdict: SHIP (deployed; production smoke test passes with 1 worker)

- **Data integrity and visual correctness:** the strongest areas. Each guarantee above is enforced by tests, and the spike is checked against the pixels actually drawn.
- **Reliability:** no open defects. Every failure path shows a visible error with Retry.
- **Performance:** the 1M-point run no longer freezes the page (longest task 332 ms at 4× throttling).
- **Acceptable limitations:** zooming to a new viewport still takes about 440 ms at 4× throttling; no screen-reader, Safari or real-device testing yet; some heavy tests are sensitive to machine load; the production smoke test times out intermittently with 4 workers.
- **Priorities:** (1) a screen-reader pass (NVDA or VoiceOver); (2) move downsampling to the worker or pre-aggregate at several resolutions, to speed up zoom; (3) Safari and real devices; (4) a shorter worker timeout or a progress signal.

AI tool usage and human verification

- **Tool: Claude Code** (Anthropic's coding agent, in VS Code; Claude Opus 5.5 in the final sessions) wrote most of the code, tests, scripts and documentation, and ran every command and browser check reported here, except the candidate's three production smoke runs. `[QUESTION FOR CANDIDATE: were any other AI tools used (e.g. Copilot, ChatGPT)? If so, name them here.]`
- **Candidate's role** (as recorded in the session): wrote the ten task specifications and their acceptance criteria; chose Vitest 4 with a Windows drive-letter wrapper over Vitest 5; set the Git workflow (work on `main`, commit and push only with approval) and merged the pull requests; deployed to Vercel and ran the first three production smoke tests. The agent committed only when asked.
- **Rejected or changed:** `npm audit fix --force` was rejected (it would have jumped to Vitest 5). Measurements overturned some of the agent's first ideas: posting a million point objects from the worker cost about 1.8 s, so typed arrays are used; an early "0 axe violations" was not trusted until manual contrast checks found the axis-line failure.
- **Executed:** all test counts and measurements above come from commands that were actually run; the logs are in `docs/qa/logs/`.
- **Personally reviewed by the candidate:** `[QUESTION FOR CANDIDATE: which diffs, screenshots or app behaviour did you review or try yourself? Do not leave this blank.]`

Reflection

[CANDIDATE TO WRITE: 4–6 sentences in your own words: what surprised you, what you would do differently, what you learned about checking AI-generated work.]